

Universidad Tecnológica Nacional

Tecnicatura Universitaria en Programación

Materia: Programación 1

Gestión de Datos de Países en Python

Proyecto Final

Integrantes:

Astargo Leonardo

Diaz Gonzalo

Fecha de Entrega: 20 de Junio de 2026

INDICE.

Marco teórico	Página 3 a 5.
Diagramas de flujo	Página 6 a 12.
Funcionamiento del programa	Página 13 a 17.
Dificultades y conclusiones	Página 18 a 20.
Links del video	Página 20.
Links repositorio GitHub	Página 20.

MARCO TEÓRICO

El presente proyecto se fundamenta en los principios de la programación estructurada y modular utilizando el lenguaje Python. A continuación, se explican de forma conceptual las herramientas sintácticas, estructuras de datos y lógica de control que permiten el funcionamiento del sistema.

1. Fundamentos de Sintaxis y Programación Secuencial

1.1 Declaración y Asignación de Variables

Una variable es un espacio reservado en la memoria de la computadora destinado a almacenar un valor que puede cambiar durante la ejecución del programa.

Python utiliza un mecanismo de **tipado dinámico**, lo que significa que no es necesario especificar si una variable es de tipo texto, entero o decimal al crearla; el lenguaje lo detecta automáticamente según el valor que se le asigne mediante el operador `=`.

1.2 Operadores Aritméticos

Son los símbolos que permiten realizar operaciones matemáticas sobre los datos numéricos. En Python se incluyen la suma (+), resta (-), multiplicación (*), división tradicional (/), división entera o de piso (// que descarta los decimales), módulo o residuo (%) que devuelve el resto de una división) y la potenciación (`^`).

1.3 Entrada y Salida de Datos

Es el canal de comunicación entre el usuario y el programa a través de la consola:

- **Entrada (input):** Detiene el programa y espera a que el usuario escriba algo con el teclado. Todo dato ingresado por este medio se recibe inicialmente en formato de texto (*string*).
- **Salida (print):** Se utiliza para mostrar mensajes, resultados o textos en la pantalla de la terminal.

1.4 Formato de Cadenas (f-strings)

Las cadenas de texto formateadas (o *f-strings*) son una herramienta que permite incrustar variables o expresiones directamente dentro de un texto de manera sencilla y ordenada. Se crean anteponiendo la letra *f* antes de las comillas (por ejemplo: `f"El país es {nombre_pais}"`), lo que facilita la lectura y evita tener que concatenar partes con el símbolo `+`.

1.5 Estructuras Secuenciales

La programación secuencial representa el flujo básico de un algoritmo. Implica que las instrucciones del código se ejecutan de manera lineal y descendente, una línea tras otra, en el orden exacto en el que fueron escritas, sin saltos ni interrupciones en el camino.

2. Estructuras de Control Repetitivas

Las estructuras repetitivas o bucles permiten ejecutar un mismo bloque de código varias veces de forma automática, evitando la duplicación de líneas.

2.1 Bucle Condicional (while)

Es una estructura que evalúa una condición lógica antes de cada iteración. Mientras esa condición sea verdadera (True), el bucle seguirá repitiendo el código en su interior. En el momento en que la condición pasa a ser falsa (False), el bucle se detiene. Es la herramienta ideal para mantener activo el menú principal del programa hasta que el usuario decida salir.

2.2 Bucle Iterativo (for)

A diferencia del anterior, el bucle for está diseñado para recorrer secuencias de datos de principio a fin (como los elementos de una lista o un rango de números). Pasa de forma ordenada por cada ítem de la colección, asignándolo a una variable de control, y finaliza automáticamente cuando ya no quedan elementos por recorrer.

3. Estructuras de Datos Compuestas y Modularización

Para organizar y manipular grandes volúmenes de información, como los registros de los países, se recurre a estructuras complejas y a la división del código.

3.1 Listas

Una lista es una colección ordenada, mutable y dinámica de elementos. Permite guardar múltiples valores bajo un solo nombre y modificarlos (agregar, quitar o editar) en cualquier momento. Cada elemento dentro de la lista ocupa una posición fija identificada por un índice numérico que comienza siempre desde el cero (0).

3.2 Diccionarios

Un diccionario es una estructura que organiza la información en pares de **clave-valor** (*key-value*). En lugar de acceder a los datos mediante un número de índice, se accede a través de una clave única (generalmente una palabra). Es ideal para representar un "objeto" o registro completo; por ejemplo, un diccionario de un país puede tener las claves "nombre", "poblacion" y "continente", facilitando el acceso directo a cada propiedad.

3.3 Funciones (Modularización)

La modularización consiste en fragmentar un programa grande en partes más pequeñas y fáciles de administrar llamadas funciones. Una función es un bloque de código aislado que realiza una tarea específica (por ejemplo, `buscar_un_pais()`). Se define una sola vez mediante la palabra `def` y puede ser ejecutada (llamada) en cualquier parte del código las veces que sea necesario, recibiendo parámetros de entrada y devolviendo un resultado final mediante la instrucción `return`.

4. Persistencia de Datos y Robustez del Software

4.1 Manejo de Archivos: Lectura desde CSV

La persistencia de datos permite que la información del programa no se borre al cerrarlo, guardándose en el disco de la computadora. Los archivos CSV (*Valores Separados por Comas*) almacenan información en forma de tabla o matriz de texto plano. Python interactúa con estos archivos abriéndolos con la función `open()` y procesándolos con el módulo especializado `csv`, convirtiendo cada fila del archivo de texto en estructuras legibles (como listas o diccionarios) dentro del programa.

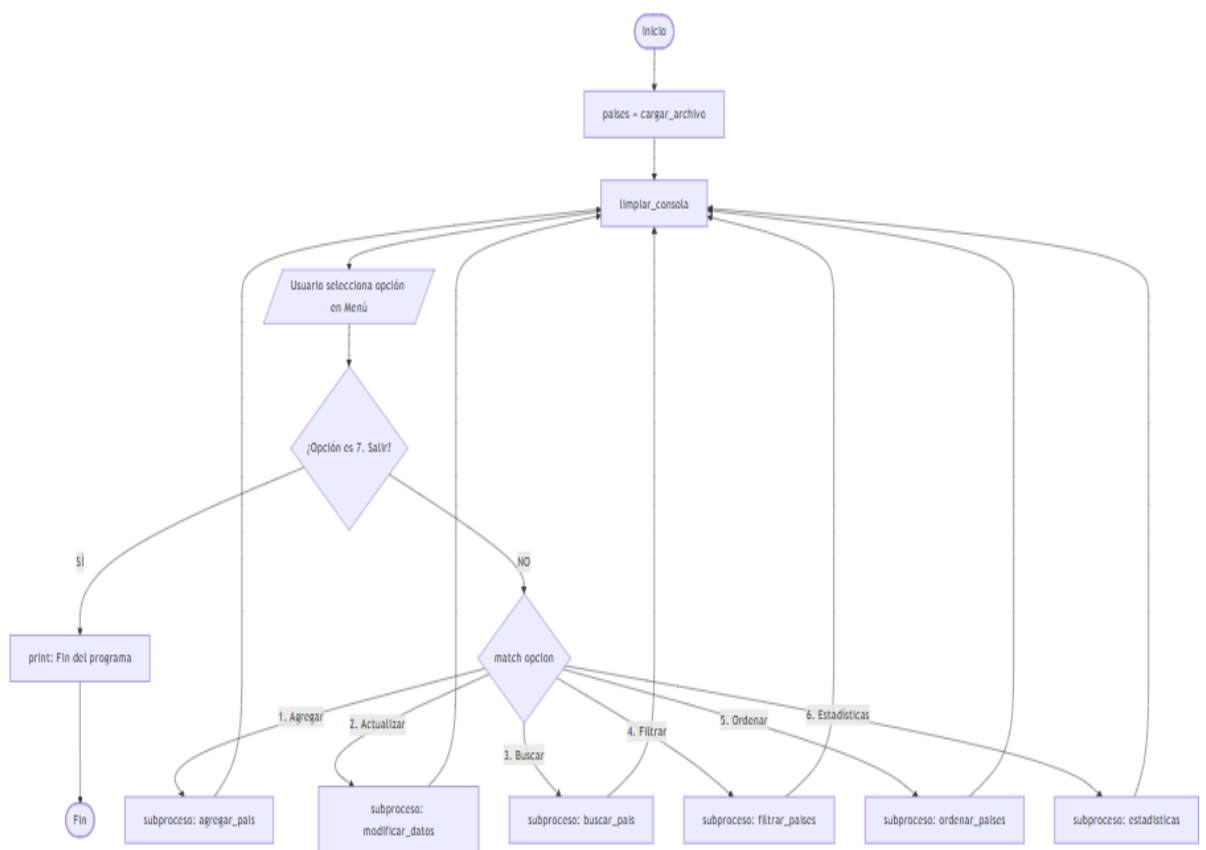
4.2 Validación de Entradas y Manejo de Errores

La robustez es la propiedad que hace que un programa sea estable y no se cierre por fallos inesperados del usuario.

- **Validación de entradas:** Consiste en revisar preventivamente lo que el usuario escribe mediante condicionales o funciones como `.isdigit()` (que chequea si el texto ingresado son solo números), asegurando que los datos cumplan con las reglas requeridas antes de ser procesados.
- **Manejo de errores (try - except):** Es una estructura de control que permite "capturar" un error en tiempo de ejecución (por ejemplo, si el usuario escribe letras cuando el programa esperaba un número para hacer un cálculo o si un archivo no existe). El código propenso a fallar se coloca en el bloque `try`, y si ocurre un fallo, la ejecución salta al bloque `except` donde se gestiona el inconveniente de manera controlada (enviando un mensaje de advertencia) en lugar de romper o congelar la terminal.

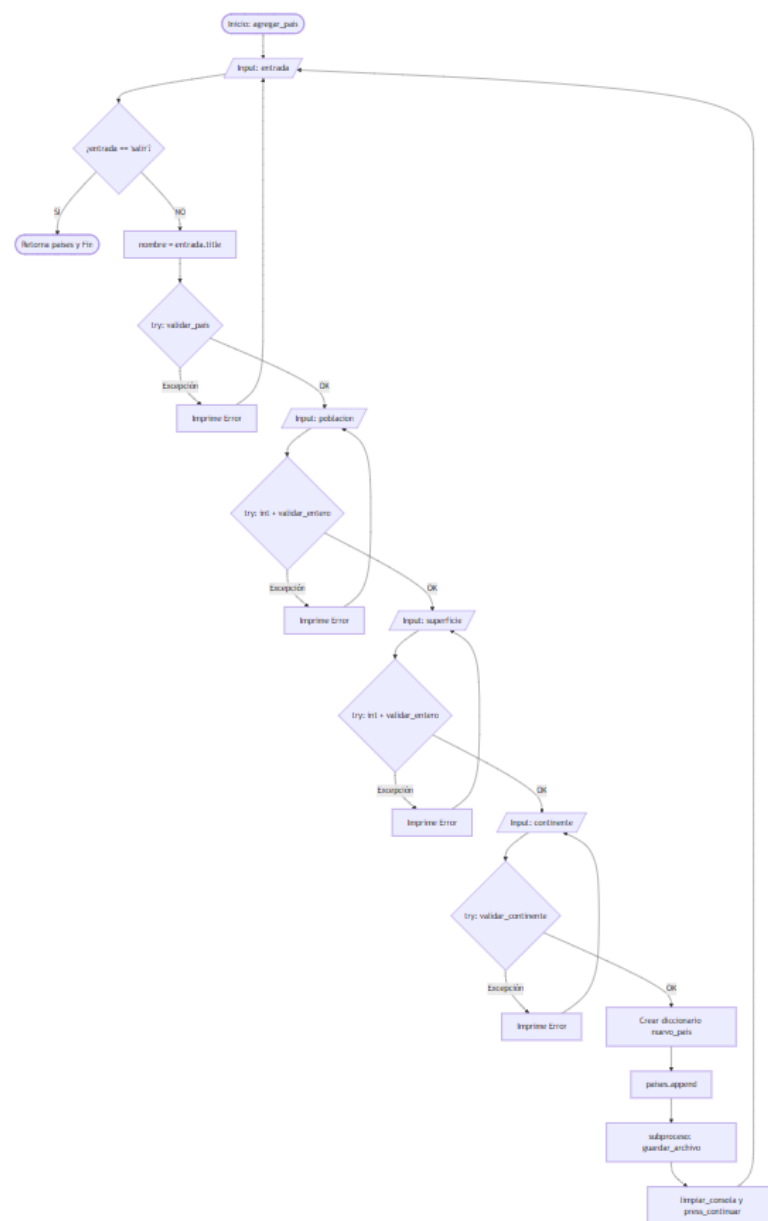
DIAGRAMA DE FLUJO DEL PROGRAMA

El siguiente diagrama de flujo representa la lógica principal del programa de gestión de datos de países. El proceso inicia con la carga de información desde un archivo CSV, continúa con la presentación de un menú interactivo y ejecuta distintas funciones según la opción seleccionada por el usuario. Luego de cada operación, el sistema retorna al menú principal hasta que se elige la opción de salida.



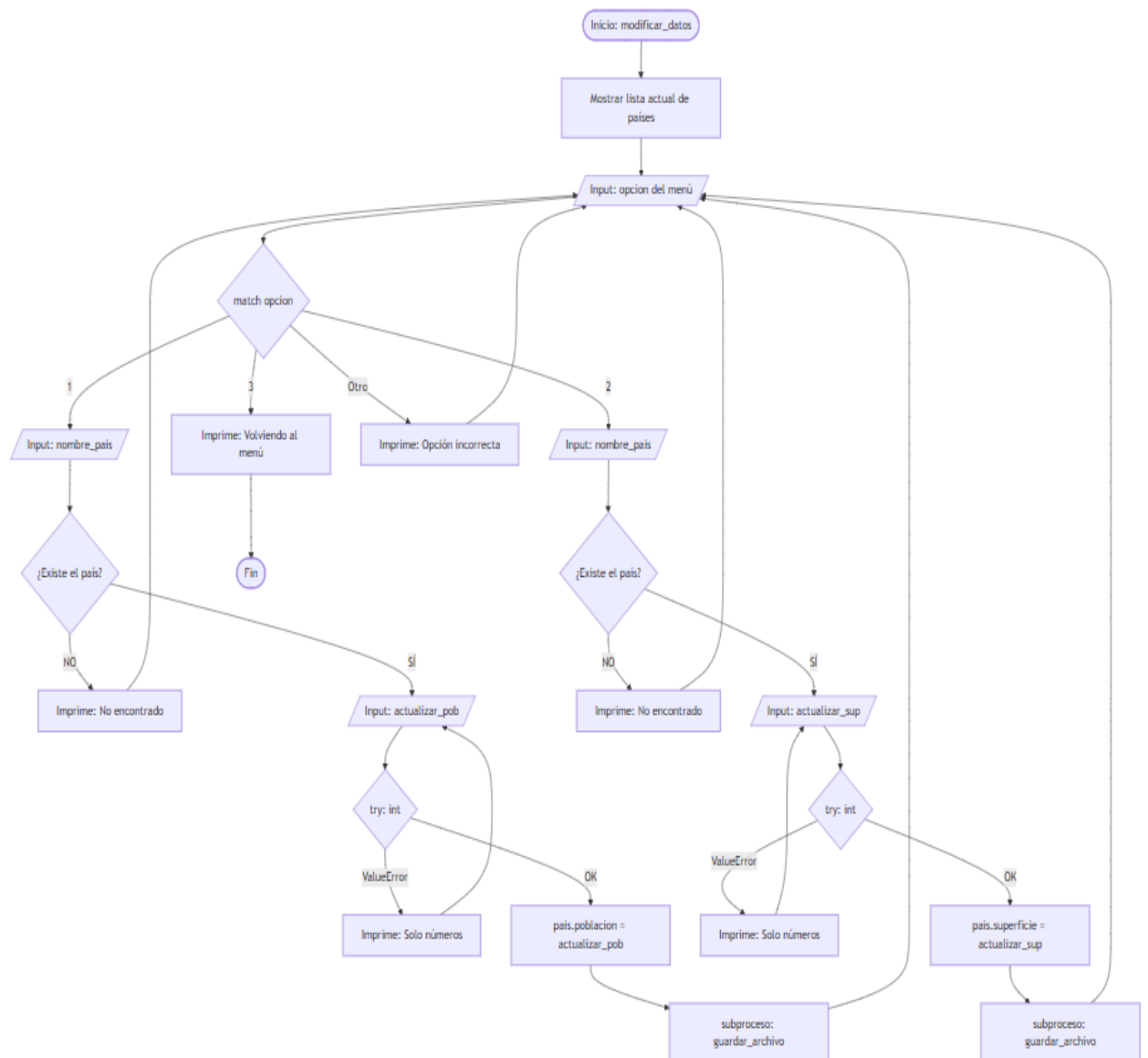
1. Agregar País (agregar_pais(países))

Este diagrama refleja el bucle de ingreso y la lógica secuencial de excepciones y validaciones que tiene tu código (error_entradavacia, error_soleletras, error_paisexiste y error_numero_negativo).



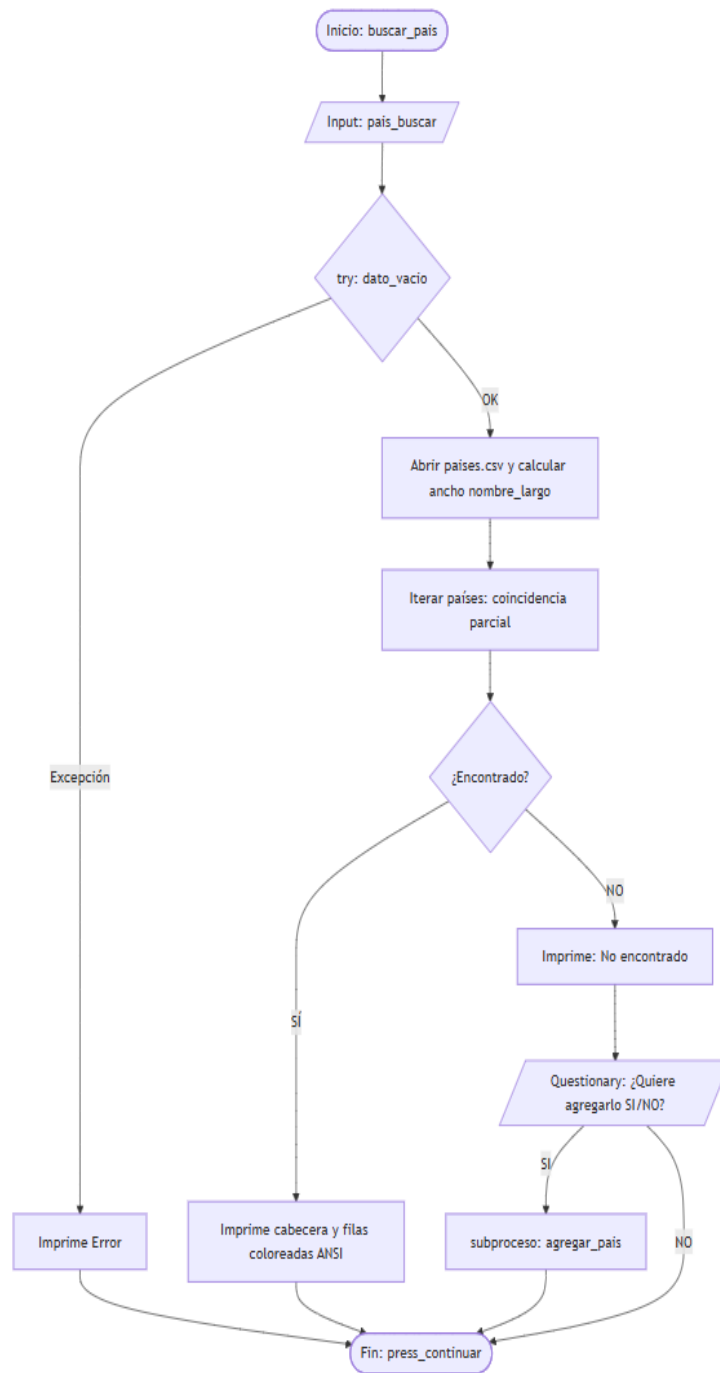
2. Actualizar Datos (modificar_datos(países))

Representa la lógica de tu archivo `modificar_datos.py`, donde primero se listan los países en pantalla y luego se entra en un menú para elegir si actualizar Población o Superficie mediante un match opción.



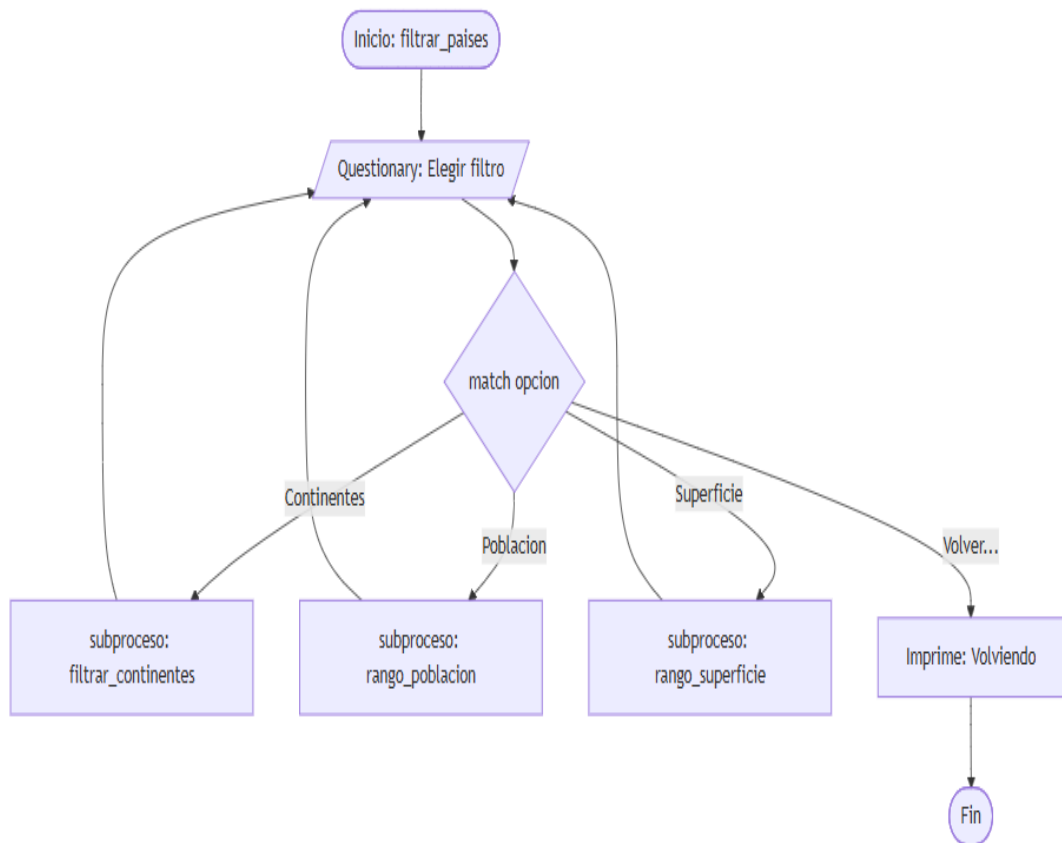
3. Buscar País por Nombre (buscar_pais(países))

Muestra la lógica de búsqueda con coincidencia parcial (recorrido del lector CSV), el formato visual con relleno y la opción interactiva de questionnaire si el país no es hallado.



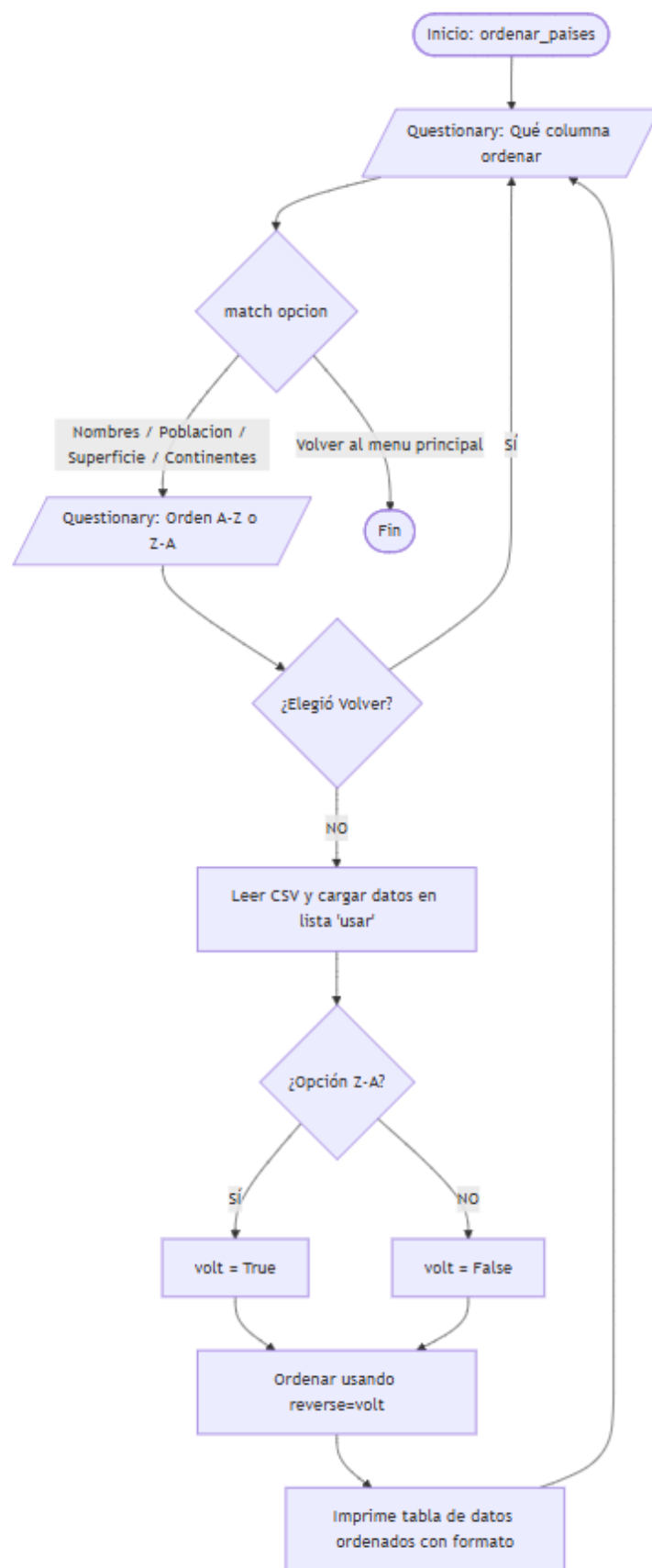
4. Filtrar Países (filtrar_paises(países))

Muestra cómo funciona la ramificación del menú de filtrado (filtrar_pais.py) que divide el flujo hacia Continentes, Población o Superficie.



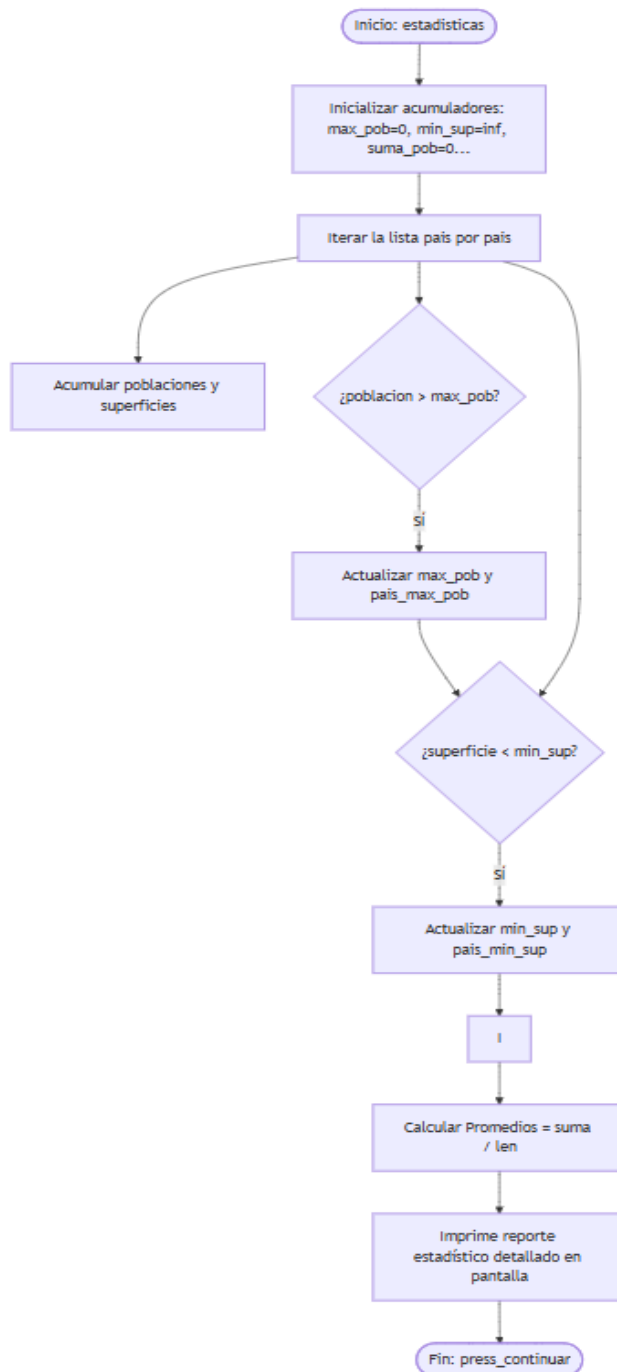
5. Ordenar Países (ordenar_paises(países))

Modela el comportamiento de ordenar_paises.py, el cual utiliza sub-menús para decidir qué columna ordenar y bajo qué criterio (Ascendente A - Z o Descendente Z - A).



6. Mostrar Estadísticas (estadisticas(países))

Aunque es una función directa, su lógica arquitectónica se basa en procesos puramente iterativos y matemáticos sobre acumuladores.



Funcionamiento y dinámica del programa.

El código del programa se encuentra dividido en una main que contiene el menú match case en donde se hace uso de funciones para dejar una visual más limpia y clara.

[illegible]

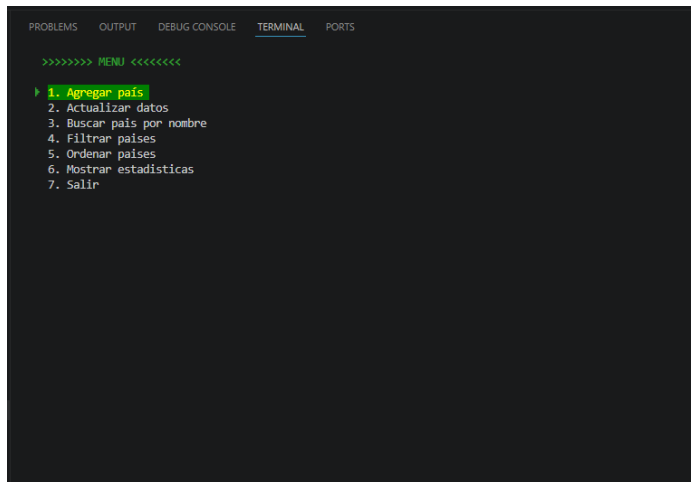
#Menu principal.

The screenshot displays a code editor interface. On the left, the 'EXPLORER' panel shows a project structure. The root is 'TPI-PROG1-26', which contains a folder named 'Funciones'. Inside 'Funciones', there is a file named '_fcn_.py' which is currently selected. Below it, a list of files is shown, including 'agregar_pais.py', 'buscar_pais.py', 'cargar_archivo.py', 'en_consola.py', 'estadisticas.py', 'excepciones.py', 'filtrar_pais.py', 'guardar_archivo.py', 'menu.py', 'modificar_datos.py', 'ordenar_paises.py', 'primera_carga.py', '.gitignore', 'main.py', 'paises.csv', and 'README.md'. On the right, the code editor shows the content of the selected file, '_fcn_.py'. It contains 12 lines of code, all starting with 'from Funciones.' followed by a function name and an asterisk, indicating imports from the 'Funciones' module. The functions listed are: 'ordenar_paises', 'buscar_pais', 'cargar_archivo', 'menu', 'agregar_pais', 'guardar_archivo', 'modificar_datos', 'filtrar_pais', 'estadisticas', 'excepciones', 'en_consola', and 'primera_carga'.

```
#Carpeta con funciones del programa.
```

Funcionamiento en consola.

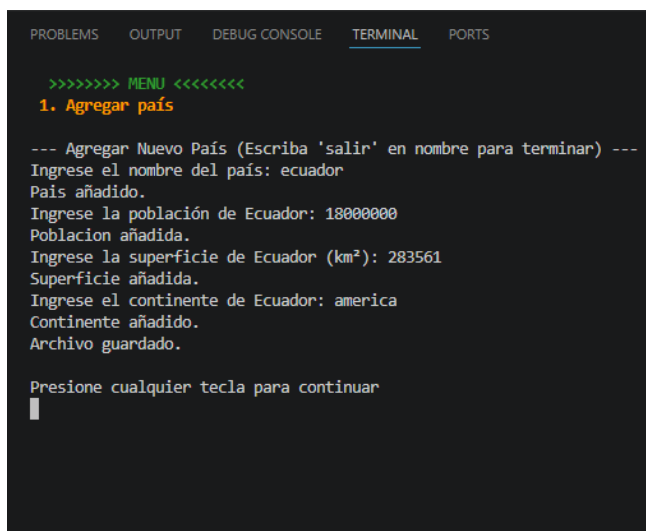
A la hora de darle funcionamiento al programa nos encontramos con un menú principal que nos da a elegir de forma interactiva la opción que deseamos utilizar.



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

>>>>>>> MENU <<<<<<<<
> 1. Agregar país
2. Actualizar datos
3. Buscar país por nombre
4. Filtrar países
5. Ordenar países
6. Mostrar estadísticas
7. Salir
```

Como primera opción tenemos la función de **agregar país** a nuestro archivo CSV. Este nos va a mostrar si queremos agregar un país o escribir salir para terminar la ejecución del menú.



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

>>>>>>> MENU <<<<<<<<
1. Agregar país

--- Agregar Nuevo País (Escriba 'salir' en nombre para terminar) ---
Ingrese el nombre del país: ecuador
País añadido.
Ingrese la población de Ecuador: 18000000
Poblacion añadida.
Ingrese la superficie de Ecuador (km²): 283561
Superficie añadida.
Ingrese el continente de Ecuador: america
Continente añadido.
Archivo guardado.

Presione cualquier tecla para continuar
█
```

La opción 2 de **actualizar datos** nos da la oportunidad de actualizar datos como la población y la superficie de un país. Esta opción necesita una coincidencia exacta en el nombre ya que se necesita acceder solo a ese país. Para ello se detalla la lista para poder revisar los países.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

País: Italia | Población: 59066225 | Superficie: 301340 | Continente: Europa
=====
País: Sudafrica | Población: 60041994 | Superficie: 1221037 | Continente: Africa
=====
País: Tanzania | Población: 61498437 | Superficie: 945087 | Continente: Africa
=====
País: Myanmar | Población: 54806012 | Superficie: 676578 | Continente: Asia
=====
País: Corea del Sur | Población: 51744876 | Superficie: 100210 | Continente: Asia
=====
País: Colombia | Población: 50882891 | Superficie: 1141748 | Continente: America
=====
País: Kenia | Población: 54985698 | Superficie: 580367 | Continente: Africa
=====
País: España | Población: 47351567 | Superficie: 505992 | Continente: Europa
=====
País: Argelia | Población: 44616624 | Superficie: 2381741 | Continente: Africa
=====
País: Sudan | Población: 44909353 | Superficie: 1861484 | Continente: Africa
=====
País: Ucrania | Población: 43733762 | Superficie: 603500 | Continente: Europa
=====
País: Irak | Población: 41179350 | Superficie: 438317 | Continente: Asia
=====
País: Afganistan | Población: 39835428 | Superficie: 652230 | Continente: Asia
=====
País: Polonia | Población: 37840001 | Superficie: 312685 | Continente: Europa
=====
País: Canada | Población: 38008007 | Superficie: 9984671 | Continente: America
=====
País: Marruecos | Población: 37344795 | Superficie: 446550 | Continente: Africa
=====
País: Arabia Saudita | Población: 35340683 | Superficie: 2149690 | Continente: Asia
=====
País: Ecuador | Población: 18000000 | Superficie: 283561 | Continente: America
=====

>>>>>>> MENU <<<<<<<
1. Actualizar poblacion.
Elija el pais que desea actualizar: canada
País: Canada | Población: 38008007
Ingresa la poblacion nueva para Canada: 38008006
Poblacion actualizada.
Archivo guardado.
>>>>>>> MENU <<<<<<<
2. Actualizar superficie.
Elija el pais que desea actualizar: canada
País: Canada | Superficie: 9984671
Ingresa la Superficie nueva para Canada: 9984672
Superficie actualizada.
Archivo guardado.
>>>>>>> MENU <<<<<<<
> 1. Actualizar poblacion.
   2. Actualizar superficie.
   3. Salir
```

La opción de **buscar país** por nombre devuelve un solo país con todos los detalles si se ingresa el nombre completo, como también devuelve l cantidad de país que encuentre en una búsqueda parcial Ej. Arg (Argentina-Argelia)

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

>>>>>>> MENU <<<<<<<
3. Buscar país por nombre
Ingresa el nombre del pais a buscar: canada

Países | Poblacion      | Superficie | Continente
-----
Canada | 38008006         | 9984672    | America

Presione cualquier tecla para continuar
█
```

Entrando a la tercera función **Filtrar país** nos encontramos con varias opciones para la búsqueda de países. Filtrando por continentes, población o superficie pidiendo valores máximos y mínimos para buscar los países que se encuentren en el rango establecido.

```

>>>>>> MENU <<<<<<<
4. Filtrar países
=== Menu para filtrar países por ===
> Continentes
  Poblacion
  Superficie
  Volver al Menu Principal

```

```

>>>>>> MENU <<<<<<<
4. Filtrar países
=== Menu para filtrar países por === Continentes

=====
MENÚ DE CONTINENTES
=====
> América
  Eruopa
  África
  Asia
  Oceanía
  Antártida
  Volver

```

```

=====
MENÚ DE CONTINENTES
=====
Volver
=== Menu para filtrar países por === Poblacion
Ingrese la población MÍNIMA: 10000000
Ingrese la población MÁXIMA: 80000000

Resultados para población entre 10,000,000 y 80,000,000:
-----
- Argentina: 45,376,764 habitantes.
- Francia: 67,391,582 habitantes.
- Tailandia: 69,950,850 habitantes.
- Reino Unido: 67,081,234 habitantes.
- Italia: 59,060,225 habitantes.
- Sudafrica: 60,041,994 habitantes.
- Tanzania: 61,498,437 habitantes.
- Myanmar: 54,806,012 habitantes.
- Corea del Sur: 51,744,876 habitantes.
- Colombia: 50,882,891 habitantes.
- Kenia: 54,985,698 habitantes.
- España: 47,351,567 habitantes.
- Argelia: 44,616,624 habitantes.
- Sudan: 44,909,353 habitantes.
- Ucrania: 43,733,762 habitantes.
- Irak: 41,179,350 habitantes.
- Afganistan: 39,835,428 habitantes.
- Polonia: 37,840,001 habitantes.
- Canada: 38,008,006 habitantes.
- Marruecos: 37,344,795 habitantes.
- Arabia Saudita: 35,340,683 habitantes.
- Ecuador: 18,000,000 habitantes.
-----

=== Menu para filtrar países por === Superficie
Ingrese la superficie MÍNIMA (km²): 10000000
Ingrese la superficie MÁXIMA (km²): 50000000

Resultados para superficie entre 10,000,000 y 50,000,000 km²:
-----
- Rusia: 17,098,242 km².
-----

=== Menu para filtrar países por ===
> Continentes
  Poblacion
  Superficie
  Volver al Menu Principal

```


Como quinta opción tenemos **ordenar países** el cual despliega un menú interactivo dando diferentes opciones de ordenamiento después dentro de cada ordenamiento da diferentes opciones para ordenar la lista de países.

```
>>>>>>> MENU <<<<<<<<
5. Ordenar países
Como quiere ordenar los datos
» . Nombres de los Países
  . Tamaño de la Poblacion
  . Tamaño de la Superficie
  . Nombre de los Continentes
  . Volver al menu principal

Como quiere ordenar los datos . Tamaño de la Poblacion
En que orden lo quiere mostrar:
» . Mayor a Menor
  . Menor a Mayor
  . Volver

Como quiere ordenar los datos . Tamaño de la Superficie
Como quiere ordenar el Tamaño de la Superficie:
» . Mayor a Menor
  . Menor a Mayor
  . Volver

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

Como quiere ordenar los datos . Nombre de los Continentes
En que orden lo quiere mostrar:
» . A - Z
  . Z - A
  . Volver

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

Como quiere ordenar los datos . Nombres de los Países
En que orden lo quiere mostrar:
» . A - Z
  . Z - A
  . Volver
```

Como última opción encontramos **mostrar estadísticas** donde solo con ingresar nos va a dar estadísticas y promedios con los datos ya cargados en el CSV.

```
>>>>>>> MENU <<<<<<<<
6. Mostrar estadísticas
=====
ESTADISTICAS
=====
-- Pais con MAYOR poblacion -- -- Pais con MENOR poblacion --
      China                      Ecuador
-- Poblacion Total --          -- Poblacion Total --
    1,411,750,000              18,000,000
=====
-- Promedio de poblacion por pais --
                157,670,793
-- Promedio de la superficie de los paises --
                2,271,090.7 km².
-- Promedio paises por continentes --
                6

Presione cualquier tecla para continuar
█
```

Dificultades y conclusiones

Una de las dificultades planteadas durante la resolución del TPI fue en la búsqueda parcial a la hora de buscar un país.

```
29 for i in lector:
30     pais = i['nombre']
31     if pais_buscar == pais[:len(pais_buscar)]:
32         corte = pais.lower().find(pais_buscar.lower()) # Aca empieza el fragmento buscado
33         sin_corte = len(pais_buscar) # El largo total del fragmento/total ingresado
34         parcial = pais[corte + sin_corte]
35         # parcial -- [: 0 + sin_corte] -- Toma de 0 hasta el fragmento/total ingresado
36         total = pais[corte + sin_corte:]
37         # total -- Toma todo lo que quede de resto si es que hay
38         coloreado = f'{Fore.GREEN}{parcial}{Fore.RESET}{total}'
39         # colorado -- Fusiona los datos ingresados anteriormente
40         espacios_relleno = " " * max(0, nombre_largo - len(pais))
41         # espacios_relleno -- Calcula cuantos espacios en blanco le tiene que agregar al ancho de fila
42         dato_total = f'{coloreado}{espacios_relleno}'
43         # dato_total -- Junta el nombre ya coloreado con los espacios de relleno
44         print(f"{dato_total:<15} | {i['poblacion']:<15} | {i['superficie']:<12} | {i['continente']:<6}")
45         encontrado = True
46     if not encontrado:
47         print(f'El pais "{Fore.RED}{pais_buscar}{Fore.RESET}" no se ha encontrado')
48         print('\n ¿Quiere agregarlo a la lista?')
```

1. La Condición de Búsqueda

`if pais_buscar == pais[:len(pais_buscar)]:`

- Verifica si el país actual **empieza** con el texto que ingresó el usuario.
- Aquí comparamos lo que escribió el usuario (`pais_buscar`) con las primeras letras del país en la base de datos, usando un recorte (slice) del mismo largo. Si coinciden, entramos a procesar ese país.

2. Encontrar y Dividir el Texto

`corte = pais.lower().find(pais_buscar.lower())`

`sin_corte = len(pais_buscar)`

- Encuentra la posición exacta donde empieza la coincidencia (en minúsculas para que no falle por mayúsculas) y mide el largo del texto buscado.
- Para no romper nada por culpa de las mayúsculas o minúsculas, pasamos todo a minúsculas temporalmente con `.lower()`. Con `.find()` averiguamos el índice exacto donde empieza la palabra buscada (`corte`) y guardamos en `sin_corte` cuántas letras mide lo que el usuario escribió.

3. El "Efecto Bisturí" (Trocear el string)

`parcial = pais[corte + sin_corte:]`

```
total = paises[corte + sin_corte:]
```

- Divide el nombre del país en dos partes: la parte que coincide y la parte que sobra.
- Acá dividimos el nombre del país en dos partes usando *slicing*. En parcial guardamos el fragmento que el usuario escribió (el que queremos colorear). En total guardamos el resto del nombre del país, es decir, lo que sobró a la derecha.

4. El Coloreado con Colorama

```
coloreado = f'{Fore.GREEN}{parcial}{Fore.RESET}{total}'
```

- Aplica el color verde usando la librería colorama.
- Fusionamos las dos partes que cortamos antes. Envolvemos la parte parcial con los comandos de Colorama para que se pinte de **VERDE** (Fore.GREEN) y luego reseteamos el color (Fore.RESET) para que el resto del nombre (total) se imprima en el color original de la consola.

5. El Ajuste de la Tabla (Formateo Estético)

```
espacios_relleno = " " * max(0, nombre_largo - len(paises))
```

```
dato_total = f'{coloreado}{espacios_relleno}'
```

- Calcula cuántos espacios faltan para que todas las columnas midan lo mismo.
- Como al agregarle códigos de color al string su longitud 'invisible' cambia y puede deformar la tabla, calculamos manualmente cuántos espacios en blanco (espacios_relleno) necesita este país para alcanzar el ancho máximo (nombre_largo). El max(0, ...) evita que de números negativos si hay un error. Al final, unimos el texto coloreado con sus espacios correspondientes.

6. La Impresión en Pantalla

```
print(f"{dato_total:<15} | {i['poblacion']:<15} | {i['superficie']:<12} |  
{i['continente']:<6}")
```

```
encontrado = True
```

- Muestra la fila alineada en la consola y activa la bandera de que hubo éxito.
-
- Finalmente, imprimimos la fila completa con un formato de columnas (:<15, :<12, etc.) para que queden perfectamente alineadas a la izquierda

con barras separadoras. Por último, marcamos la variable encontrado = True para indicarle al programa que la búsqueda tuvo éxito y no muestre un mensaje de 'país no encontrado' al final.

Ya no como problemática si no para darle un nivel mas profesional usamos las api: coloreto(Colores en el menú, estadísticas, etc.), questionnaire(menú interactivos), tqdm(Para realizar la barra de carga que se muestra al iniciar) estas fueron investigadas con las referencias que nos dieron en clase y la IA para entender la forma en la que podíamos integrar al programa.

Reflexión final.

Aprendimos como es el trabajo colaborativo usando GitHub por primera vez, aprender a repartir tareas y luego tratar de darle el funcionamiento al programa integrando los dos trabajos. Así mismo creemos que es algo en lo que tenemos que continuar trabajando para reforzar los conocimientos y habilidades.

Adquirimos funciones nuevas como, limpiar la consola, tipos de menú interactivo, buscando la opción deseada con flechas (questionary), aplicamos colores al menú buscando salir de la estética básica que veníamos trabajando en clase.

Links repositorio de GitHub

<https://github.com/gonzadiaz993/TPI-PROG1-26>

Links Video explicativo

<https://www.youtube.com/watch?v=ABu342zW6KM>